

Do Time-Series Foundation Models Need Fine-Tuning? Wind Power Forecasting with PatchTST and Chronos

DATA612 PWS1 Project Proposal

Simran Kharbanda
(UID: 122283671)

Tanmay Sharma
(UID: 121997234)

1 Background and Problem Significance

Background. Transformers have become one of the main architectures for long-horizon forecasting. Self-attention connects every time step to every other one, so a single layer can pick up long-range patterns like daily cycles. PatchTST made transformers competitive on the standard benchmarks by cutting each series into patches that act as tokens, and by modelling each variable on its own [1]. Pretrained foundation models go further: Chronos scales and quantizes values into a fixed vocabulary of tokens and trains T5 models (8M to 710M parameters) on roughly 84 billion observations from 28 public datasets, so it can forecast series it has never seen [2]. Such models are adapted to a new domain by fine-tuning, either of every weight or, with LoRA, of only a low-rank update BA to the frozen weights W_0 , so that $W = W_0 + BA$ with rank $r \ll \min(d, k)$ [3]. In practice it’s still unclear which option works best for a given domain: a pretrained model used as is, a fine-tuned one, or a small transformer trained from scratch. That is the question this project tests.

Significance. Wind power is hard to plan for because turbine output depends on wind speed, and wind speed changes minute to minute. Grid operators rely on short-term forecasts to schedule reserves and keep supply and demand balanced, and forecast errors mean either wasted reserve capacity or shortfalls. Most public wind datasets only include a few turbines and leave out their locations and internal status. SDWPF instead provides 134 turbines with locations and turbine readings [4]. The accuracy-versus-compute trade-off we measure helps teams choose between using a foundation model off the shelf, fine-tuning it, or training a small model themselves.

2 Problem Statement

We forecast each turbine’s active power (Patv) at 10-minute resolution: given the past $L = 288$ steps (two days), we learn f with $\hat{y}_{t+1:t+H} = f(y_{t-L+1:t})$ for $H = 6, 36$ and 64 steps (1 h, 6 h and about 10.7 h). We train one model shared across all 134 turbines rather than one model per turbine. We cap H at 64 because Chronos was trained with a 64-step forecast length and its accuracy tends to drop beyond that, which would disadvantage it for reasons unrelated to our questions. We address three research questions:

RQ1 Does our PatchTST, trained from scratch on SDWPF, outperform Chronos used zero-shot?

RQ2 How much of any gap between the two do full fine-tuning and LoRA fine-tuning of Chronos close?

RQ3 How does each approach change when trained on 5%, 25% and 100% of the training data, and what does each cost in GPU hours and trainable parameters?

3 Training Data

We use the Spatial Dynamic Wind Power Forecasting (SDWPF) dataset, released by Baidu for the KDD Cup 2022 wind power forecasting challenge [4]: 134 turbines from one wind farm, recorded every 10 minutes for 245 days (4,727,520 records), with active power (Patv, kW) as the target. It can be downloaded from Baidu AI Studio after registration, or from the Hugging Face mirror `aigrids/WindFarm_raw`.

SDWPF isn’t in the Chronos training data (Table 3 of [2]), but some other wind and solar datasets are. So for Chronos, this particular farm is new, but the domain isn’t. We split the 245 days in time order, with no shuffling, into training, validation and test periods of 70%, 10% and 20% (about 171, 25 and 49 days); the validation period is used for early stopping and hyperparameter tuning. For RQ3, the 5% and 25% subsets are the most recent part of the training period. We take the most recent data rather than a random sample to mirror the realistic deployment case of a newly commissioned farm with only a short recent history, so RQ3 measures performance under limited recent history, not data quantity alone. Following [4], negative Patv is set to zero; readings with $\text{Patv} \leq 0$ while wind speed exceeds 2.5 m/s, or with any blade pitch angle above 89° , are *unknown*, and readings with nacelle direction outside $[-720^\circ, 720^\circ]$ or wind direction outside $[-180^\circ, 180^\circ]$ are *abnormal*. Missing, unknown and abnormal readings are excluded from scoring, and the rest form the valid set V (Section 7). Chronos receives missing values as NaN, which it handles natively; for PatchTST and the LSTM, we fill gaps by carrying the last value forward. SDWPF doesn’t come with a published license, so we only use it for non-commercial academic work and don’t redistribute the raw files.

4 Approach

PatchTST: We implement PatchTST in PyTorch from the paper [1], using the official code only as a reference to check against. Each input window is normalized per channel, cut into patches of length 16 with stride 8 (36 patches for $L = 288$, with end padding), embedded, passed through a transformer encoder, and mapped to the forecast by a linear quantile head trained with quantile loss, whose median serves as the point forecast.

Chronos: We use Chronos-T5-small (46M parameters), and Chronos-T5-base (200M) if compute allows; we chose the original Chronos because its training-data list is published and excludes SDWPF [2]. We apply it (a) zero-shot, (b) with full fine-tuning via the repository’s `scripts/training/train.py`, after converting SDWPF to the script’s Arrow input format, and (c) with LoRA on the T5 attention weights through `peft` [6], which we add ourselves since the repository has no LoRA support.

Baselines and grid: The baselines are persistence (repeat the last value), seasonal naive (same time on the previous day) and an LSTM. The grid $\{\text{PatchTST, Chronos full fine-tuning, Chronos LoRA}\} \times \{5\%, 25\%, 100\% \text{ of training data}\} \times \{3 \text{ seeds}\}$, with zero-shot Chronos as a fixed reference that needs no training. PatchTST’s linear head is tied to a fixed forecast length, so we consider two designs: one model with a 64-step head scored at all three horizons, or a separate model per horizon. We choose between them with a single-seed pilot on the validation set at full data. Together with Chronos full fine-tuning, LoRA, the LSTM baseline (full data, 3 seeds) and the ETTh1 check, this gives roughly 40–60 training runs, which we estimate at 50–100 A100 GPU hours, to be confirmed by the pilot. All methods are scored with the same evaluation code.

5 Team Responsibilities

Member	Responsibilities
Simran Kharbanda	PatchTST implementation in PyTorch and check against the official repo; multi-GPU (DDP) fine-tuning of Chronos-T5-base; spatial-context extension.
Tanmay Sharma	Data download, cleaning, splits and subsets; persistence, seasonal-naive and LSTM baselines; Chronos zero-shot eval and full fine-tuning; shared evaluation code and metrics.
Both	Chronos LoRA fine-tuning with <code>peft</code> ; Zaratan environment and Slurm job scripts, experiment grid, analysis and plots, interim report, presentation, final report.

6 Framework and Compute

All models use PyTorch, with Hugging Face `transformers` and `peft` [6] and the Apache-2.0 `chronos-forecasting` package [5] for Chronos. We build on two existing implementations: the official PatchTST code is only used as a reference, since our model is re-implemented, and the Chronos training script is used with SDWPF

converted to its Arrow input format and is extended with LoRA support. Training runs on UMD’s Zaratán HPC cluster through Slurm, with development on our laptops and Google Colab, and code shared through Git. Every run has a fixed seed and its own configuration file, and the results are logged per run (Weights & Biases or a CSV).

7 Validation and Evaluation Metrics

We compute every metric on the test period, separately for each horizon H , and only over the valid readings V (Section 3). Forecasts are issued at a fixed interval across the test period (e.g. every 6 hours), and every model is scored on the same windows. Point accuracy is measured by MAE and RMSE in kW and by MASE. MASE falls below 1 when a model does better than the seasonal-naive forecast ($m = 144$, one day; its denominator is computed on the training series). Probabilistic accuracy is the weighted quantile loss (WQL) over $Q = \{0.1, 0.2, \dots, 0.9\}$, produced by Chronos and the quantile-head PatchTST. For cost we track GPU hours, the number of trainable parameters, and inference time per forecast. To verify the pipeline, our PatchTST must reproduce the paper’s ETTh1 error within 5% before we use it on SDWPF; every setting is run with 3 seeds and reported as mean $\pm$ standard deviation; and the baselines use the same evaluation code. For RQ1, we look at the difference in MAE and WQL between PatchTST and zero-shot Chronos with the full training data. For RQ2, we look at how much of that difference full fine-tuning and LoRA fine-tuning each close. For RQ3, we plot error against the fraction of training data used and compare those curves with the cost of each approach.

$$\text{MAE} = \frac{1}{N} \sum_{t \in V} |y_t - \hat{y}_t|, \quad \text{RMSE} = \sqrt{\frac{1}{N} \sum_{t \in V} (y_t - \hat{y}_t)^2}, \quad \text{MASE} = \frac{\text{MAE}}{\frac{1}{T-m} \sum_{t=m+1}^T |y_t - y_{t-m}|}, \quad (1)$$

$$\rho_q(y, \hat{y}) = (y - \hat{y})(q - \mathbf{1}[y < \hat{y}]), \quad \text{WQL} = \frac{1}{|Q|} \sum_{q \in Q} \frac{2 \sum_{t \in V} \rho_q(y_t, \hat{y}_t^{(q)})}{\sum_{t \in V} |y_t|}. \quad (2)$$

8 Potential Extensions

If time and compute allow, we will add spatial context: wind reaching one turbine soon reaches its neighbours, and SDWPF records each turbine’s position [4], which Chronos cannot use because it forecasts one series at a time [2]. We will concatenate the patch tokens of each turbine and its $k = 3\text{--}5$ nearest neighbours into one encoder sequence, so attention can relate patches across turbines, and test at which horizons this beats plain PatchTST and Chronos. A smaller alternative is a LoRA rank sweep ($r = 4, 8, 16, 32$) relating accuracy to the number of trainable parameters.

9 Risks and Fallbacks

If zero-shot Chronos beats every trained model, that is still a valid answer to RQ1, and the RQ3 curves remain informative. If LoRA conflicts with the Chronos wrapper, we apply LoRA directly to the underlying T5 model, or else fine-tune only its last few layers.

References

- [1] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. *ICLR*, 2023.
- [2] A. F. Ansari et al. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024. <https://arxiv.org/abs/2403.07815>
- [3] E. J. Hu et al. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022.
- [4] J. Zhou et al. SDWPF: A dataset for spatial dynamic wind power forecasting over a large turbine array. *Scientific Data*, 11, 2024. <https://arxiv.org/abs/2208.04360>
- [5] Amazon Science. chronos-forecasting (software, Apache-2.0). <https://github.com/amazon-science/chronos-forecasting>
- [6] Hugging Face. PEFT: Parameter-efficient fine-tuning (software). <https://github.com/huggingface/peft>